	Mezcla de Lógica e Interfaz
1.	No puede estar en ninguna ventana la suma de 0.15 por la bolsa cuando el pedido es para llevar. Tiene que estar en la clase Pedido bien directamente en el método que calcula el importe o bien en otro método aparte que haga este incremento.
2.	No puede haber invocaciones a JOptionPane en ninguna clase de lógica. Si fuera necesario notificar al usuario el resultado de la invocación a un método de la capa de lógica, se modifica el tipo de retorno del método donde esté esa invocación y ya en la ventana, en función del valor devuelto, se muestra el mensaje correspondiente.
3.	Esto es, en general: no puede haber ningún import de javax.swing ni de java.awt en las clases de la lógica.
4.	No puede haber recorridos de listas en las ventanas, ni sumas, ni restas….ni cualquier otra operación que sea para modificar algún atributo de alguna clase de negocio. También se recomienda que todas las operaciones de comparación para habilitación/deshabilitación, pasar de pantalla, etc. estén recogidas en la capa de negocio.

	Filtro por tipo de artículo
1.	5 métodos en VentanaPrincipal: optimizar para que sea sólo uno que reciba el tipo de artículo por el que se quiere filtrar
2.	El recorrido del arrayList de artículos para obtener el array filtrado tiene que estar en la clase Carta y ofrecerse desde la clase McDonalds.
3.	No hace falta un contador para dimensionar el array de articulos filtrados. El método .toArray pasa la lista auxiliar de artículos filtrados a un array
4.	Además, en ningún caso deben contarse los elementos que hay de cada tipo en el fichero para dimensionar el array que se debe devolver. Si metemos, por ejemplo, alguna hamburguesa más en el fichero se produciría un arrayIndexOutOfBoundsException.
5.	No es necesario buscar un substring ni un char para filtrar: para eso está precisamente el atributo "tipo" del artículo
6.	No es necesario definir 4 listas auxiliares para el filtro; con una es suficiente
7.	Tras filtrar se debe ver la imagen correcta, la correspondiente al primer elemento del combo. Para esto no es necesario volver a cargar la imagen; con hacer getCbArticulos().setSelectedIndex(0) tras establecer el modelo es suficiente ya que el actionPerformed del combo ya carga la imagen